

App Dev Project Report

1. Student Details

Name: Debankush Ghosh

Roll Number: 24f3001469

Email: 24f3001469@ds.study.iitm.ac.in

About Me:

I am a student of the IIT Madras BS Degree Program with strong interests in full-stack development, Python backend architecture, and database-driven applications. I enjoy building real-world software systems that automate workflows and improve user experience through structured design, role-based access, and secure backend implementation.

2. Project Details

Project Title:

Hospital Management System Web Application

Problem Statement:

To develop a role-based hospital management system that replaces manual paperwork and simplifies day-to-day operations.

The system must enable patients to:

- register and log in securely
- book appointments with selected specialists
- view their medical booking history

The system must also allow doctors to:

- log in separately
- view all patients and their medical conditions
- manage appointments effectively

The application should maintain data consistency, ensure secure authentication, and provide a simplified interface for both patients and doctors.

Approach:

The application is built using the **Flask** web framework with a backend-first development approach.

Core approaches used:

- **Role-Based Access Control (RBAC):**
Patients and doctors have separate login flows and separate permissions.
- **Secure authentication:**
Passwords are hashed using Werkzeug before storing in the SQLite database.
- **Relational database design:**
Used SQLAlchemy ORM to define Users and Appointments models, ensuring structured data storage and easy querying.
- **Dynamic UI rendering:**
Jinja2 templates handle conditional rendering based on user role and session state.
- **Billing and Appointment Modules:**
Implemented appointment booking, viewing history, and a billing form stored in server memory for demonstration.

3. AI/LLM Declaration

I used Google Gemini / ChatGPT to assist in:

- writing SQLAlchemy models
- improving the responsive CSS of the appointment and billing pages
- debugging minor Jinja2 errors
- formatting parts of documentation

AI/LLM usage was limited to **10–15%**, mostly for stylistic or syntax suggestions.

All main logic, debugging, testing, API development, and database integrations were implemented manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Routing, sessions, request handling
SQLAlchemy	Object Relational Mapping (ORM) for database interactions
Werkzeug Security	Password hashing & verification
SQLite	Lightweight local database for storing users & appointments
Jinja2	Templating engine for dynamic HTML rendering

HTML5 / CSS3

Frontend structure and custom UI design

Python 3

Application logic, server-side programming

5. Database Schema / ER Diagram

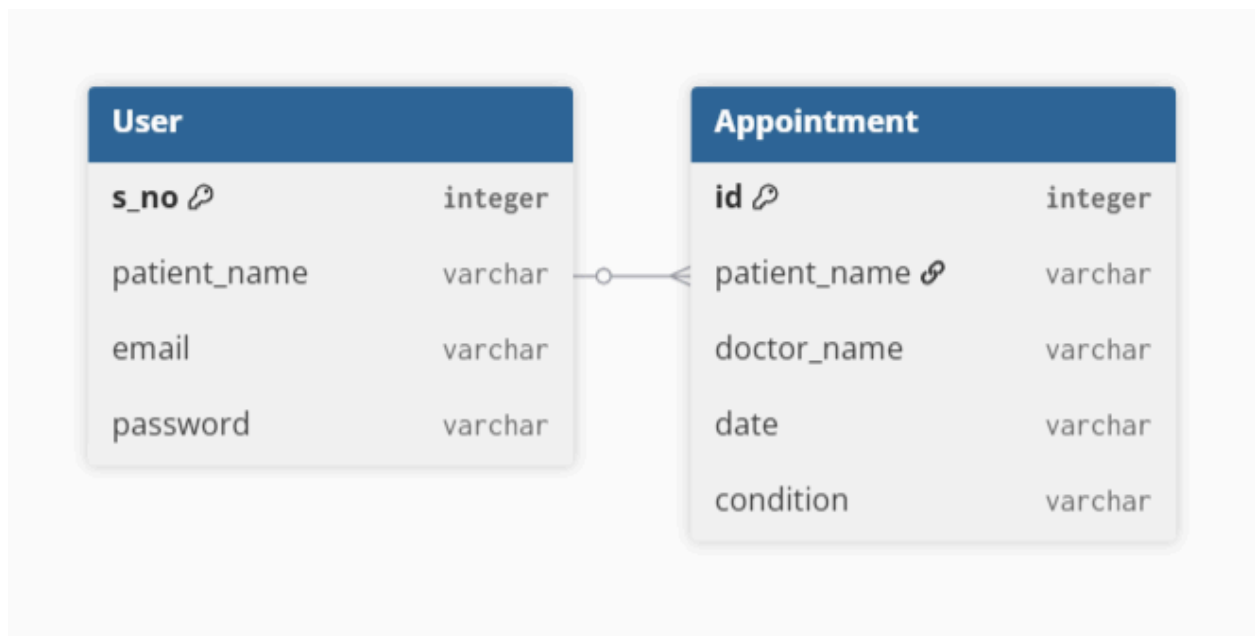
Relationships:

- **One-to-Many (logical mapping):**
`User.patient_name` → `Appointment.patient_name`
(Linked via name field in this implementation)

ER Diagram:

Created using dbdiagram.io

Link: <https://dbdiagram.io/d/691dffe7228c5bbc1a98ef25>



6. API Resource Endpoints

Endpoint	Method	Description
<code>/signup</code>	POST	Register a new patient

<code>/login</code>	POST	Authenticate patient and create a session
<code>/doctor_login</code>	POST	Doctor-only login
<code>/appointment</code>	POST	Create an appointment
<code>/appointment</code>	GET	Fetch appointments of logged-in user
<code>/billing</code>	POST	Submit billing details
<code>/patients</code>	GET	Doctor view: all patients + conditions
<code>/logout</code>	GET	Logout and clear session

(Your project does not have delete/edit routes now; you can add later.)

7. Architecture and Features

Architecture Overview

- **app.py**
The main application entry point containing:
 - Flask routes
 - SQLAlchemy models
 - session handling
 - business logic for appointments and billing
- **/templates**
Contains all HTML frontend pages (login, signup, appointment, doctor dashboard, etc.) using Jinja2.
- **/static**
(Optional) Custom CSS files and images.
- **user.db**
SQLite database storing:
 - registered users
 - appointment records